

Structured Data Integration (1)

TECHNOLOGIES FOR INFORMATION SYSTEMS

Letizia Tanca

Politecnico di Milano



Recall: Designing data integration in the Multidatabase, with a unique data model

1. Source schema identification (when present)
2. Source schema reverse engineering (data source conceptual schemata)
3. Conceptual schemata integration and restructuring: conflict resolution and restructuring
4. Conceptual to logical translation (of the obtained **global schema**)
5. Mapping between the global logical schema and the single schemata (**logical view definition**)
6. After integration: **query-answering** through data views

Mapping between the global logical (mediated) schema and the single source schemata.

In general:

- Formally, a data integration system is a triple
(G, S, M)

Where: **G** is the global schema, **S** is the set of sources and **M** is the set of mappings.

- A query to the integrated system is posed in terms of G and specifies which data of the **virtual** database we are interested in.

- Thus the problem is:

understanding which real data (in the data sources) correspond to those virtual data.

NOTE: THIS PROBLEM CONCERNS THE CASE OF VIRTUAL INTEGRATION SINCE, WITH MATERIALIZED INTEGRATION, THE USERS QUERY A PHYSICAL DATABASE

Mapping between the global logical (mediated) schema and the single source schemata (logical view definition)

- Two basic techniques:
 - **GAV (Global As View)**
 - **LAV (Local As View)**
- Note that later on we'll learn that:
Similar basic techniques are adopted *also in the case of different data models*, but then we have to add
model transformation techniques

GAV (Global As View)

- Up to now we supposed that the global schema *be derived* from the integration process of the data source schemata
- Thus the global schema is *expressed in terms of the data source schemata*
- This approach is called the *Global As View approach*

The other possible ways

LAV (Local As View)

- The global schema has been designed **independently of** the data source schemata
- The relationship (mapping) between sources and global schema is obtained **by defining each data source as a view over the global schema**

GLAV (Global and Local As View)

- The relationship (mapping) between sources and global schema is obtained by defining a set of views, **some over the global schema and some over the data sources**

GAV

A GAV mapping is a set of assertions, one for each element g of the global schema $\mathbf{G}$

$$g \rightarrow q_s$$

That is, the mapping (view) specifies g as a query q_s over the data sources. This means that

the mapping tells us exactly how g is computed

- OK for stable data sources
- Difficult to extend with a new data source

GAV example

SOURCE 1

Product(Code, Name, Description, Warnings, Notes, CatID)

Category(ID, Name, Description)

Version(ProductCode, VersionCode, Size, Color, Name,
Description, Stock, Price)

SOURCE 2

Product(Code, Name, Size, Color, Description, Type, Price, Q.ty)

Type(TypeCode, Name, Description)

note: here we do not care about data types...

SOURCE 1

Product(Code, Name,
Description, Warnings, Notes,
CatID)

Version(ProductCode,
VersionCode, Size, Color,
Name, Description, Stock,
Price)

SOURCE 2

Product(Code, Name, Size, Color,
Description, Type, Price, Q.ty)

GLOBAL SCHEMA

CREATE VIEW GLOB-PROD AS
SELECT Code AS PCode, VersionCode
as VCode, Version.Name AS Name,
Size, Color, Version.Description as
Description, CatID, Price, Stock

FROM SOURCE1.Product,
SOURCE1.Version

WHERE Code = ProductCode

UNION

SELECT Code AS PCode, null as
VCode, Name, Size, Color,
Description, Type as CatID, Price,
Q.ty AS Stock

FROM SOURCE2.Product

Query processing in GAV

QUERY OVER THE GLOBAL SCHEMA

```
SELECT PCode, VCode, Price, Stock  
FROM GLOB-PROD  
WHERE Size = "V" AND Color = "Red"
```

Important note: here, we do not take care of the **keys**, because in order to reconcile the keys we should apply **keygen** and for the moment this is irrelevant

In this particular case the transformation is easy, since the combination operator is a UNION →
push selections through union!!

```
SELECT Code, VersionCode, Price, Stock  
FROM SOURCE1.Product, SOURCE1.Version  
WHERE Code = ProductCode AND Size = "V" AND Color = "Red"  
UNION  
SELECT Code, null, Price, Q.ty  
FROM SOURCE2.Product  
WHERE Size = "V" AND Color = "Red"
```

Further considerations on GAV

- Suppose we introduce a new source
- The simple views we have just created must be modified
- In the simplest case (like in the GLOB_PROD case) we only need to add a union with a new SELECT-FROM-WHERE clause
- This is not true in general, since view definitions may be much more complex
- In LAV this problem does not occur

LAV

A mapping LAV is a set of assertions, one for each element s of each source S

$$s \rightarrow q_G$$

Thus the content of each source is characterized in terms of a view q_G over the global schema

- OK if the global schema is stable, e.g. based on a domain description (e.g. an ontology*) or an enterprise model
- It favours extensibility
- Query processing much more complex

(*) No problem if you don't know what an ontology is, we'll study it later on, when we discuss other forms of integration

The same example, in LAV

SOURCE 1

Product(Code, Name, Description, Warnings, Notes, CatID)

Version(ProductCode, VersionCode, Size, Color, Name, Description, Stock, Price)

SOURCE 2

Product(Code, Name, Size, Color, Description, Type, Price, Q.ty)

GLOBAL SCHEMA (in this case, already defined)

GLOB-PROD (PCode,VCode, Name, Size, Color, Description, CatID, Price, Stock)

We have to express the source tables
as views over the global schema

GLOBAL SCHEMA

GLOB-PROD (PCode, VCode, Name, Size, Color, Description, CatID, Price, Stock)

easy for SOURCE 2:

SOURCE 2

Product (Code, Name, Size, Color, Description, Type, Price, Q.ty)

CREATE VIEW SOURCE2.Product AS

SELECT PCode AS Code, Name, Size, Color, Description, CatID
as Type, Price, Stock AS Q.ty

FROM GLOB-PROD

What about SOURCE 1 ???

GLOBAL SCHEMA

GLOB-PROD (PCode, VCode,
Name, Size, Color,
Description, CatID, Price,
Stock)

SOURCE 1

Product(Code, Name,
Description, Warnings, Notes,
CatID)

Version(ProductCode,
VersionCode, Size, Color,
Name, Description, Stock,
Price)

•

```
CREATE VIEW SOURCE1.Product AS
SELECT Pcode AS Code,
Name?,Description?,
Warnings ?, Notes?, CatID
FROM GLOB-PROD
```

% in the global schema, the
attributes here in green belong to
the VERSION table, so where do
we find the ones belonging to the
PRODUCT table ?

```
CREATE VIEW SOURCE1. Version AS
SELECT Pcode AS ProductCode,
VCode as VersionCode, Size, Color,
Name, Description , Stock, Price)
FROM GLOB-PROD
```

•

Notes

Some information is lacking!!! there is no corresponding value for Warnings, Notes, etc... !!!

NO PROBLEM !!!

...no query coming from the global schema will ever require those attributes, since they are not present in the global schema

Query processing in LAV

- Information is in the sources, while the mapping, i.e. the view definition, specifies the **opposite transformation** w.r.t. the one we need !
- Not possible to obtain answers by a simple, obvious or direct computation of the query definition
- No simple rule or view unfolding for the relations in the body as in GAV
- We need to produce a plan, i.e., a rewriting of the query as a **set of queries to the sources** and a prescription of how to **combine their answers**
- Both sources are implicitly needed: the system must extract the necessary information from **SOURCE1.VERSION** and from **SOURCE2.PRODUCTS**

Query processing in LAV

The queries are expressed in terms of the global schema, therefore the needed transformation is the inverse of the defined views:

some REASONING is needed

```
SELECT PCode, VCode, Price, Stock
FROM GLOB-PROD
WHERE Size = "V" AND Color = "Red"
```

The views specify transformations from global to sources, thus:

```
CREATE VIEW SOURCE1. Version AS
```

```
SELECT Pcode AS ProductCode, VCode as VersionCode, Size, Color, Name,
       Description, Stock, Price
FROM GLOB-PROD
```

Do the same for **SOURCE 2.Product**

Another example: 2 tables from two different sources DB1 and DB2, but these are not composed by a union → we need unfolding

DB1: R1 (A, B, C)

DB2: R2 (B, D)

Global Query:

Select R1.A

From R1

Where R1.B in

(Select R2.B From R2)

Query over DB2:

Select R2.B into X

From R2

Query over DB1:

Select R1.A

From R1

Where R1.B in X

GAV

Global As View

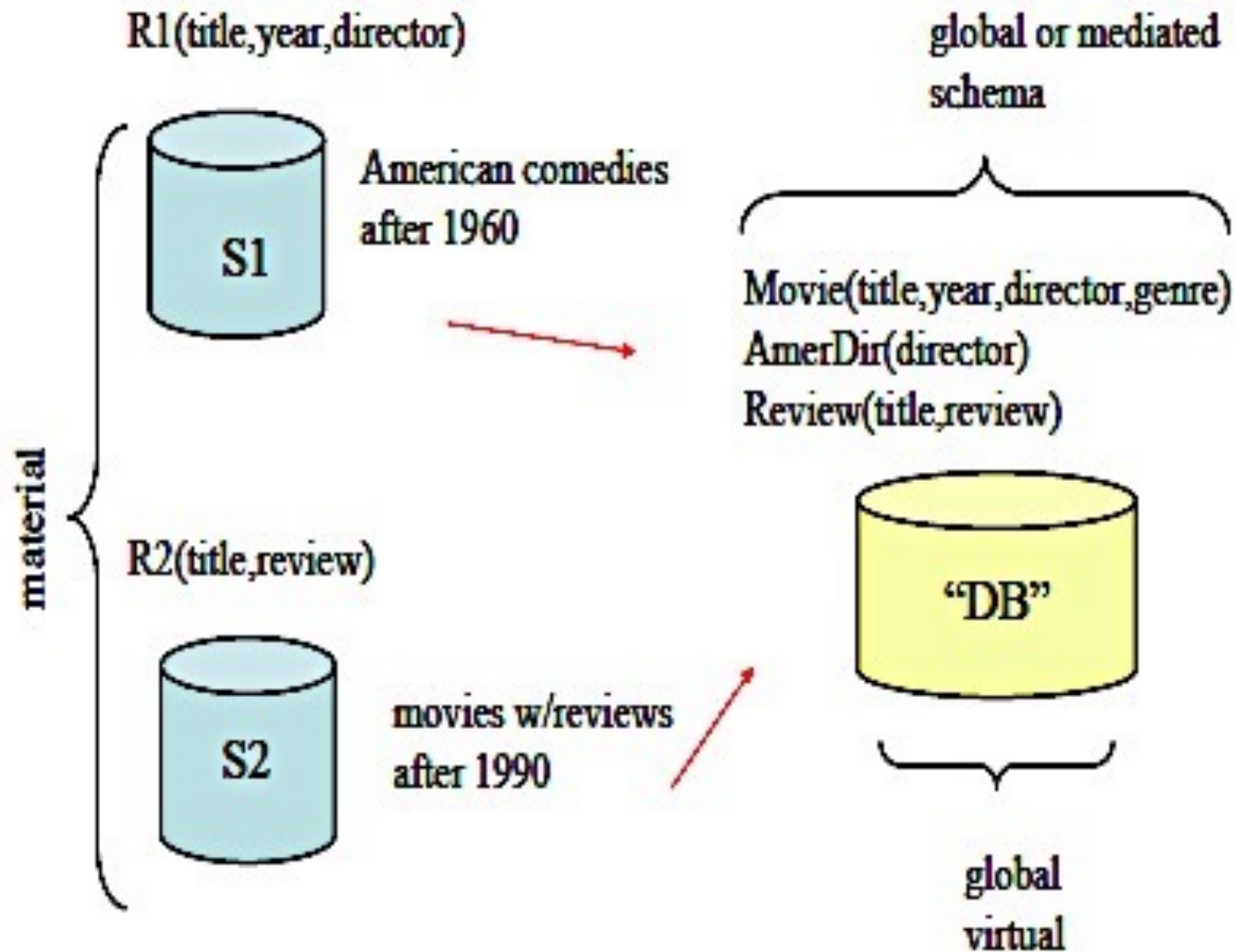
- Mapping quality depends on how well we have compiled the sources into the global schema through the mapping
- Whenever a source changes or a new one is added, the global schema needs to be reconsidered

LAV

Local As View

- Mapping quality depends on how well we have characterized the sources
- High modularity and extensibility (if the global schema is well-designed, when a source changes or is added, only its definition is to be updated)
- Query processing needs reasoning

LAV Example (Bertossi)



Example (Bertossi)

Global schema G:

Movie(Title, Year, Director, Genre),

AmerDir (Director),

Review(Title, Rev)

With LAV, the relations of sources S1, S2 are defined as views over the global schema, thus:

S1: has R1, containing comedies, filmed after 1960, with American directors and the year they came out. View:

$R1(\text{Title}, \text{Year}, \text{Director}) \leftarrow \text{Movie}(\text{Title}, \text{Year}, \text{Director}, \text{Genre}),$
 $\text{AmerDir}(\text{Director}), \text{Genre} = \text{comedy}, \text{Year} \geq 1960.$

S2: has R2, containing movies filmed after 1990 with their reviews, but no directors. View:

$R2(\text{Title}, \text{Rev}) \leftarrow \text{Movie}(\text{Title}, \text{Year}, \text{Director}, \text{Genre}),$
 $\text{Review}(\text{Title}, \text{Rev}), \text{Year} \geq 1990.$

Query processing in LAV

Query over G: ?? All comedies with their reviews filmed since 1950 ??

$Ans (Title, Rev) \leftarrow Movie(Title, Year, Director, "comedy"), Review(Title, Rev), Year \geq 1950$

- The query is rewritten in terms of the views and can be computed like this:

1. Extract values for Title from R1 (it contains *the comedies*)
2. Extract all the tuples from R2 (it contains *the reviews*)
3. At the global level, **compute the join via Title**

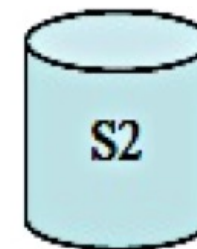
Due to the limited contents of the sources, we only obtain comedies by American directors filmed after 1990 with their reviews

R1(title,year,director)



American comedies
after 1960

R2(title,review)



movies w/reviews
after 1990

Source incompleteness

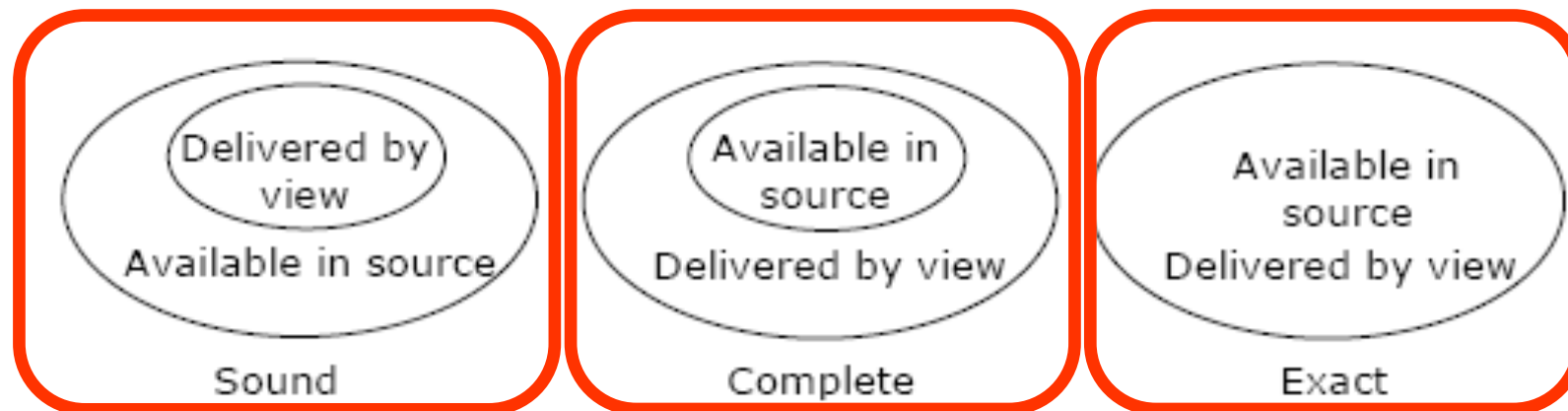
- Besides the info in **S2**, there might exist other information about movies before **1990** with their reviews, not contained in **S2**
- Thus, the information provided by **S2** is somehow “incomplete” wrt what **G** “expresses” (and might contain)
- In general, a data source might contain only a part of the information of the same kind “expected” in the global system
- This is the most common scenario: sources are assumed to be *incomplete* w.r.t. the global schema

Sound, exact, and complete mappings

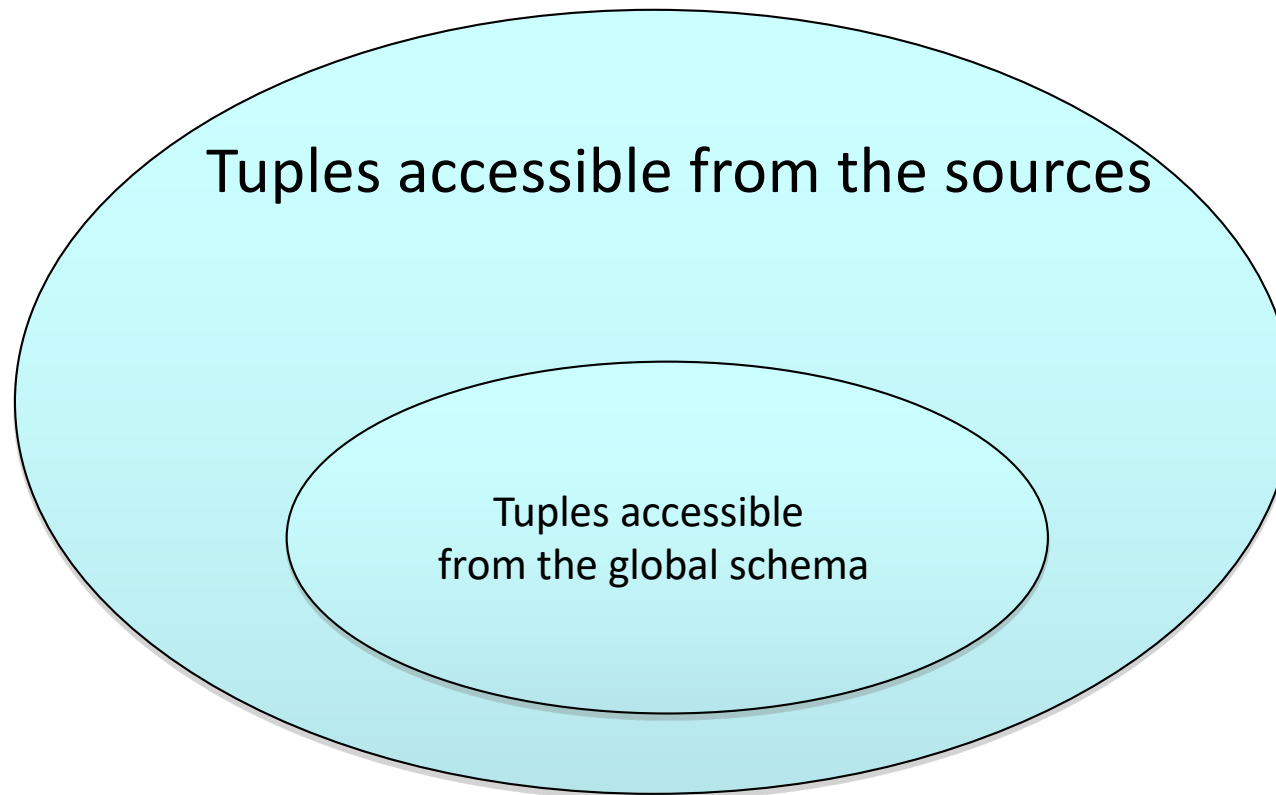
A mapping defined over some data source is sound when it provides a subset of the data that is available in the data source that corresponds to the definition.

A mapping is complete if it provides a superset of the available data in the data source that corresponds to the definition.

A mapping is exact if it provides all and only data corresponding to the definition: it is both sound and complete



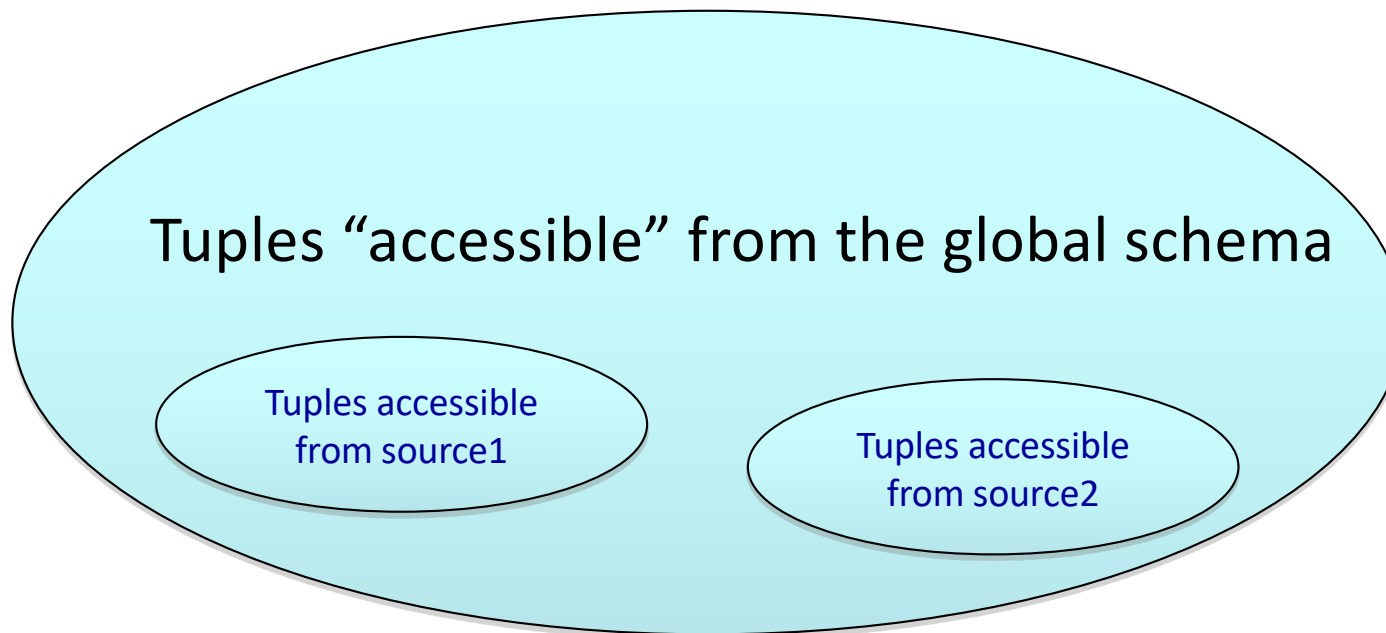
GAV with integrity constraints



The mapping can be **exact** or **only sound**

LAV approach

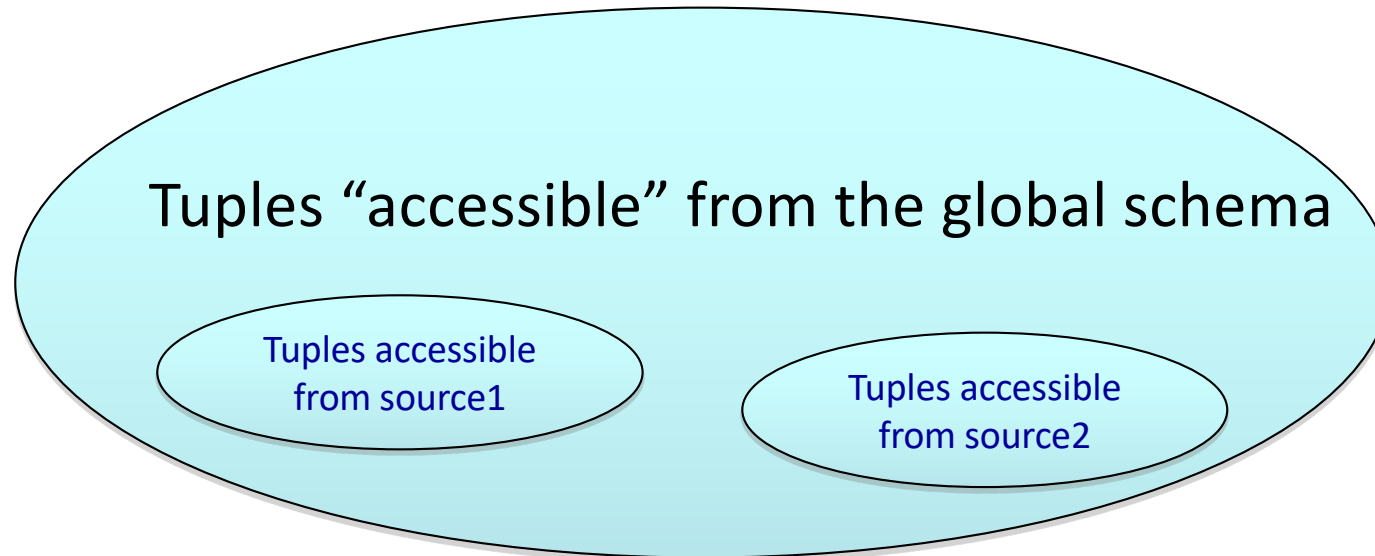
The global schema can be defined
independently of the sources' schemata



The mapping can be **exact** or **only complete** like in **S2 of the GAV example**, which contained **products but not versions**, i.e. only a part of the information of the same kind in the global system

Careful !!!

The word “complete” may arise confusion



- In the LAV case, the mapping can be exact or only complete, due to the incompleteness of one or more sources !!
- The mapping is complete because the global schema covers the source contents, but the sources are incomplete because they do not cover the data as “expected” from the global schema, which has been defined independently
- of the source contents!

The most useful (and used) integration operators to write relational GAV views

- Union (sources must have the same schema)
- Outerunion
- Join
- Outerjoin
- Generalization

OPERATORS: EXAMPLES

R1

SSN	NAME	AGE	SALARY
123456789	JOHN	34	30K
234567891	KETTY	27	25K
345678912	WANG	39	32K

R2

SSN	NAME	SALARY	PHONE
234567891	KETTY	20K	1234567
345678912	WANG	22K	2345678
456789123	MARY	34K	3456789

R1 OU R2

SSN	NAME	AGE	SALARY	PHONE
123456789	JOHN	34	30K	NULL
234567891	KETTY	27	25K	NULL
345678912	WANG	39	32K	NULL
234567891	KETTY	NULL	20K	1234567
345678912	WANG	NULL	22K	2345678
456789123	MARY	NULL	34K	3456789

R1 JOIN R2

SSN,NAME

SSN	NAME	AGE	SALARY1	SALARY2	PHONE
234567891	KETTY	27	25K	20K	1234567
345678912	WANG	39	39K	22K	2345678

OPERATORS: EXAMPLES (2)

R1

SSN	NAME	AGE	SALARY
123456789	JOHN	34	30K
234567891	KETTY	27	25K
345678912	WANG	39	32K

R2

SSN	NAME	SALARY	PHONE
234567891	KETTY	20K	1234567
345678912	WANG	22K	2345678
456789123	MARY	34K	3456789

R1 OJ R2

SSN	NAME	AGE	SALARY	PHONE
123456789	JOHN	34	30K	NULL
234567891	KETTY	27	??	1234567
345678912	WANG	39	??	2345678
456789123	MARY	NULL	34K	3456789

R1 Ge R2

SSN	NAME	SALARY
234567891	KETTY	??
345678912	WANG	??
123456789	JOHN	30K
456789123	MARY	34K

What shall we do about these “uncertain” values ?

Bibliography

- A. Doan, A. Halevy and Z. Yves, Principles of Data Integration, Morgan Kaufmann, 2012
- L. Dong, D. Srivastava, Big Data Integration, Morgan & Claypool Publishers, 2015
- Roberto De Virgilio, Fausto Giunchiglia, Letizia Tanca (Eds.): Semantic Web Information Management – A Model-Based Perspective. Springer 2009, ISBN 978-3-642-04328-4
- M. Lenzerini, Data Integration: A Theoretical Perspective, Proceedings of ACM PODS, pp. 233-246, ACM, 2002, ISBN: 1-58113-507-6
- Clement T. Yu, Weiyi Meng, Principles of Database Query Processing for Advanced Applications , Morgan Kaufmann, 1998, ISBN: 1558604340